Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №2
«Tracerout»

Подготовила: Студент гр. 410902
Кухто В.С.

Проверила: Болтак С.В.

Минск 2026

Условие задания: Необходимо разработать простейший аналог утилиты traceroute / tracert, изученной в рамках первой лабораторной работы. В качестве протокола передачи данных можно использовать ICMP или UDP (на выбор). При реализации использовать программный интерфейс сокетов (готовые реализации отправки/получения echo запросов использовать запрещено).

Утилита должна принимать в качестве параметра IP-адрес целевого узла и выводить узлы маршрута аналогично системному traceroute / tracert (порядковый номер, время ожидания ответа для каждого отправленного пакета и адрес узла). Следует отправлять более одного пакета.

Проверьте, что разработанная программа дает такие же результаты, как и системная. Проанализируйте трафик, генерируемый вашей программой в процессе работы, с помощью Wireshark. Также Wireshark можно использовать при отладке, в случае проблем с получением эхо-ответов.

Обратите внимание на значение контрольной суммы (Wireshark показывает, верно ли оно подсчитано) и обновление значения sequence number между отправками (изучите, как оно изменяется в системной утилите traceroute / tracert).

Дополнительное задание

Добавить параметр командной строки, активирующий режим разрешения в имена узлов через обратные DNS-запросы. В этом режиме помимо IP-адресов узлов маршрута должны выводиться и их имена (такое поведение является поведением по умолчанию для системных tracert / traceroute).

Добавить возможность задавать имя целевого узла вместо его IP-адреса: mytraceroute google.com.

Выполнение работы:

Написание утилиты tracert было выполнено на языке Python. Разработка происходила в среде Anaconda, приложении Spyder. Запустить написанную утилиту можно командой mytracert 8.8.8.8, где mytracert – исполняемый bat-файл.

На рисунке 1 можно увидеть, как работает написанная мной утилита:

```
D:\ksis\lr>mytracert 8.8.8.8
Трассировка маршрута к 8.8.8.8 [8.8.8.8]
с максимальным числом прыжков 30:

 1       2 ms       3 ms       2 ms  192.168.1.1
 2      52 ms      32 ms      40 ms  172.18.129.66
 3       *       *       *  Превышен интервал ожидания для запроса.
 4       *       *       *  Превышен интервал ожидания для запроса.
 5      52 ms      33 ms      45 ms  195.50.15.58
 6      48 ms      33 ms      45 ms  185.11.76.62
 7      36 ms      43 ms      45 ms  185.11.76.28
 8      38 ms      49 ms       *  74.125.49.252
 9      59 ms      58 ms      61 ms  192.178.97.15
10       *       *       *  Превышен интервал ожидания для запроса.
11      42 ms      39 ms      39 ms  8.8.8.8

Трассировка завершена.
```

Рисунок 1

Как видно на рисунке 1, с помощью написанной мной утилиты tracert можно отследить маршрут отправленных icmp-пакетов (по три пакета на каждом прыжке, как у системной утилиты tracert).

Для сравнения на рисунке 2 представлено использование системной утилиты tracert.

```
D:\ksis\l 2>tracert 8.8.8.8

Трассировка маршрута к dns.google [8.8.8.8]
с максимальным числом прыжков 30:

  1     1 ms    <1 мс     1 ms  ik41.home [192.168.1.1]
  2    37 ms    33 ms    36 ms  172.18.129.66
  3    36 ms    47 ms    26 ms  172.18.129.72
  4    43 ms    67 ms    28 ms  195.50.15.59
  5    50 ms    30 ms    47 ms  195.50.15.58
  6    41 ms    36 ms    38 ms  185.11.76.62
  7    31 ms    35 ms    51 ms  185.11.76.28
  8     *        *       53 ms  74.125.49.252
  9    51 ms    40 ms    38 ms  192.178.97.15
 10    54 ms    41 ms    51 ms  216.239.40.43
 11    52 ms    38 ms    36 ms  dns.google [8.8.8.8]

Трассировка завершена.
```

Рисунок 2

Сравнив результат работы написанной мной утилиты и уже существующей, можно сделать вывод, что они работают аналогично. На каждом прыжке можно увидеть адрес маршрутизатора (например, в обоих случаях на первом прыжке виден адрес моего основного шлюза (192.168.1.1), а на конечном прыжке – цель с адресом 8.8.8.8.

Запуская сетевой анализатор трафика WireShark можно увидеть, что контрольная сумма для пакетов считается корректно (рисунок 3). Для подсчета контрольной суммы была создана функция calculate_checksum(data) (исходный код присутствует в этой же папке в моем репозитории):

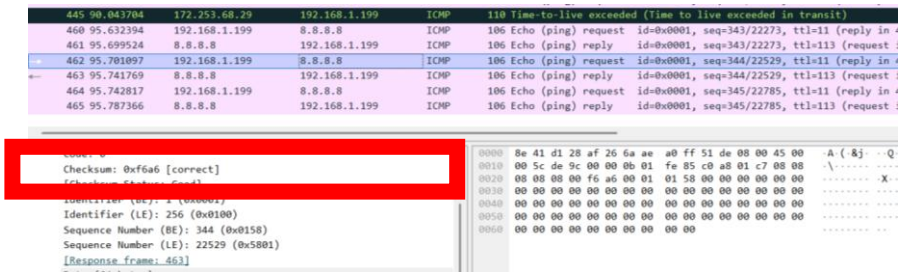
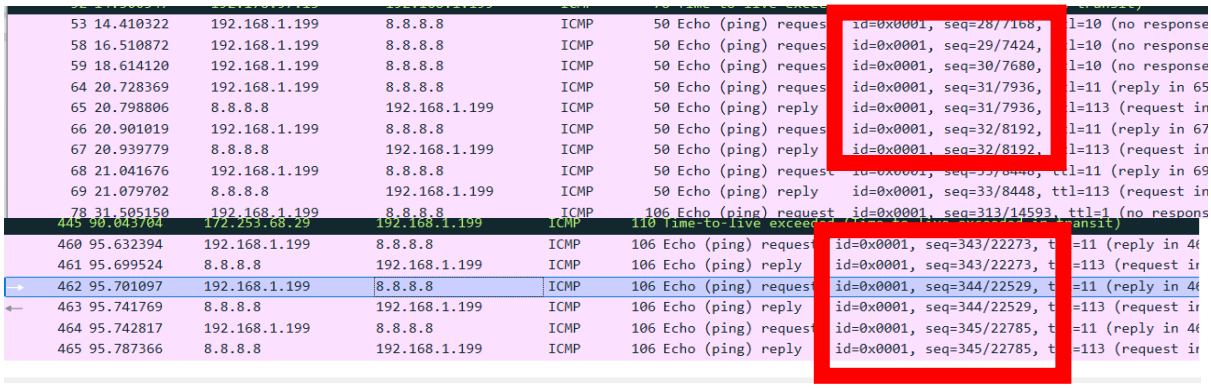


Рисунок 3

На рисунке 3.1 можно увидеть, что sequence number считается аналогично существующей утилите (сверху – результат захвата трафика вследствие применения моей написанной утилиты, снизу – системной).

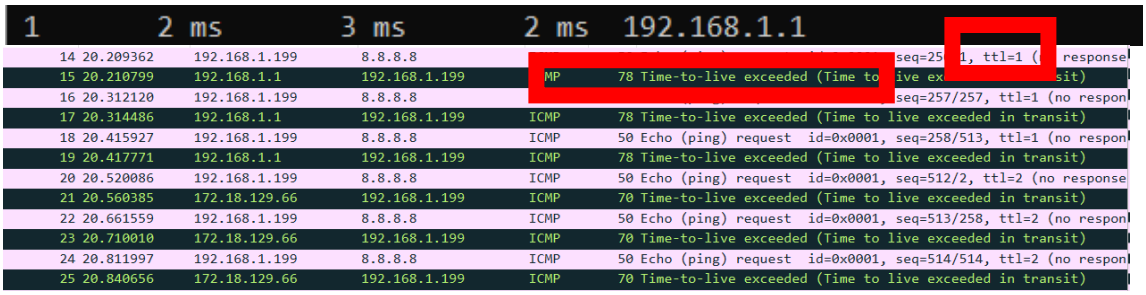


53	14.410322	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) reques id=0x0001, seq=28/7168, l=10 (no response
58	16.510872	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) reques id=0x0001, seq=29/7424, l=10 (no response
59	18.614120	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) reques id=0x0001, seq=30/7680, l=10 (no response
64	20.728369	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) reques id=0x0001, seq=31/7936, l=11 (reply in 65
65	20.798806	8.8.8.8	192.168.1.199	ICMP	50	Echo (ping) reply id=0x0001, seq=31/7936, l=113 (request in
66	20.901019	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) reques id=0x0001, seq=32/8192, l=11 (reply in 67
67	20.939779	8.8.8.8	192.168.1.199	ICMP	50	Echo (ping) reply id=0x0001, seq=32/8192, l=113 (request in
68	21.041676	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) request ttl=11 (reply in 69
69	21.079702	8.8.8.8	192.168.1.199	ICMP	50	Echo (ping) reply id=0x0001, seq=33/8448, ttl=113 (request in
78	31.505150	192.168.1.199	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=313/14593, ttl=1 (no respons
445	90.043704	172.253.68.29	192.168.1.199	ICMP	110	Time-to-live exceed ansit)
460	95.632394	192.168.1.199	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=343/22273, t =11 (reply in 46
461	95.699524	8.8.8.8	192.168.1.199	ICMP	106	Echo (ping) reply id=0x0001, seq=343/22273, t =113 (request in
462	95.701097	192.168.1.199	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=344/22529, t =11 (reply in 46
463	95.741769	8.8.8.8	192.168.1.199	ICMP	106	Echo (ping) reply id=0x0001, seq=344/22529, t =113 (request in
464	95.742817	192.168.1.199	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=345/22785, t =11 (reply in 46
465	95.787366	8.8.8.8	192.168.1.199	ICMP	106	Echo (ping) reply id=0x0001, seq=345/22785, t =113 (request in

Рисунок 3.1

Также на приведенном выше скриншоте видно, что идентификатор пакета отображается аналогично системной утилите

Проанализируем захваченный трафик. Как видно на рисунке 4, на первом прыжке было отправлено 3 icmp-пакета. TTL для них соответствует значению 1. В ответ на эти пакеты можно увидеть пакеты с сообщением «Time-to-live exceed», это означает, что пакет дошел до маршрутизатора, там время жизни пакета стало равным 0 и пакет был уничтожен, однако конечная цель еще не достигнута.

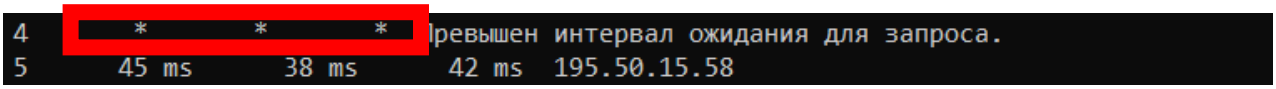


```
1        2 ms        3 ms        2 ms  192.168.1.1
```

14	20.209362	192.168.1.199	8.8.8.8			seq=25 1, ttl=1 (response
15	20.210799	192.168.1.1	192.168.1.199	MP	78	Time-to-live exceeded (Time to ive ex sit)
16	20.312120	192.168.1.199	8.8.8.8			seq=257/257, ttl=1 (no respon
17	20.314486	192.168.1.1	192.168.1.199	ICMP	78	Time-to-live exceeded (Time to live exceeded in transit)
18	20.415927	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) request id=0x0001, seq=258/513, ttl=1 (no respon
19	20.417771	192.168.1.1	192.168.1.199	ICMP	78	Time-to-live exceeded (Time to live exceeded in transit)
20	20.520086	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) request id=0x0001, seq=512/2, ttl=2 (no response
21	20.560385	172.18.129.66	192.168.1.199	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
22	20.661559	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) request id=0x0001, seq=513/258, ttl=2 (no respon
23	20.710010	172.18.129.66	192.168.1.199	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
24	20.811997	192.168.1.199	8.8.8.8	ICMP	50	Echo (ping) request id=0x0001, seq=514/514, ttl=2 (no respon
25	20.840656	172.18.129.66	192.168.1.199	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

Рисунок 4

В командной строке на некоторых прыжках можно увидеть символ * вместо адреса маршрутизатора, однако это не означает, что он не работает, так как далее передача пакетов не останавливается. Это может означать, что маршрутизатор сильно загружен в это время или не настроен на icmp-пакеты (рисунок 5).



```
4        *        *        *  Превышен интервал ожидания для запроса.
5     45 ms     38 ms     42 ms  195.50.15.58
```

Рисунок 5

В WireShark этому пакету соответствуют пакеты без ответа (рисунок 6):

35	36.490985	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1026/516, ttl=4 (no respo
38	39.600894	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1280/5, ttl=5 (no respons
39	39.644634	195.50.15.58	192.168.1.199	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
40	39.746337	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1281/261, ttl=5 (no respo
41	39.784089	195.50.15.58	192.168.1.199	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
42	39.885364	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1282/517, ttl=5 (no respo
43	39.927097	195.50.15.58	192.168.1.199	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
44	40.030144	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1536/6, ttl=6 (no respons
45	40.063910	185.11.76.62	192.168.1.199	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
46	40.165300	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=1537/262, ttl=6 (no respo

Рисунок 6

Также можно увидеть пакеты, на которые приходит ответ с узла 8.8.8.8 (рисунок 7).

82	54.035073	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=2816/11, ttl=11 (reply in
83	54.070812	8.8.8.8	192.168.1.199	ICMP	50 Echo (ping) reply id=0x0001, seq=2816/11, ttl=114 (request
84	54.172655	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) id=0x0001, seq=2817/267, ttl=11 (reply i
85	54.210968	8.8.8.8	192.168.1.199	ICMP	50 Echo (ping) reply id=0x0001, seq=2817/267, ttl=114 (reques
86	54.312818	192.168.1.199	8.8.8.8	ICMP	50 Echo (ping) request id=0x0001, seq=2818/523, ttl=11 (reply i
87	54.350429	8.8.8.8	192.168.1.199	ICMP	50 Echo (ping) reply id=0x0001, seq=2818/523, ttl=114 (reques

Рисунок 7

В написанной мной утилите можно увидеть прыжок под номером 11, где ответ приходит от узла 8.8.8.8 (рисунок 8).

```
D:\ksis\lr2>mytracert 8.8.8.8
Трассировка маршрута к 8.8.8.8 [8.8.8.8]
с максимальным числом прыжков 30:

 1      2 ms      3 ms      2 ms  192.168.1.1
 2     41 ms     49 ms     29 ms  172.18.129.66
 3      *         *         *     Превышен интервал ожидания для запроса.
 4      *         *         *     Превышен интервал ожидания для запроса.
 5     45 ms     38 ms     42 ms  195.50.15.58
 6     34 ms     40 ms     38 ms  185.11.76.62
 7     35 ms     37 ms     40 ms  185.11.76.28
 8     42 ms     44 ms      *     74.125.49.252
 9     61 ms     48 ms     43 ms  192.178.97.15
10      *         *         *     Превышен интервал ожидания для запроса.
11     36 ms     39 ms     38 ms  8.8.8.8

Трассировка завершена.
```

Рисунок 8

Дополнительно был реализован параметр командной строки, активирующий режим разрешения в имена узлов через обратные DNS-запросы, в этом режиме помимо IP-адресов узлов маршрута выводятся и их имена (рисунок 9).

```
D:\ksis\ r2>mytracert 8.8.8.8 -d
Трассировка маршрута к 8.8.8.8 [8.8.8.8]
с максимальным числом прыжков 30:

 1       2 ms       3 ms       3 ms  192.168.1.1 [ik41.home]
 2      33 ms      39 ms      39 ms  172.18.129.66
 3       *       *       *  Превышен интервал ожидания для запроса.
 4       *       *       *  Превышен интервал ожидания для запроса.
 5      74 ms      30 ms      37 ms  195.50.15.58
 6      50 ms      42 ms      51 ms  185.11.76.62
 7      70 ms      32 ms      40 ms  185.11.76.28
 8       *      40 ms      39 ms  74.125.49.252
 9      58 ms      43 ms      39 ms  192.178.97.15
10       *       *       *  Превышен интервал ожидания для запроса.
11      55 ms      29 ms      43 ms  8.8.8.8 [dns.google]

Трассировка завершена.
```

Рисунок 9

Также была добавлена возможность задавать имя целевого узла вместо его IP-адреса (рисунок 10).

```
D:\ksis\l 2>mytracert skillbox.ru
Трассировка маршрута к skillbox.ru [185.65.148.253]
с максимальным числом прыжков 30:

 1       2 ms       3 ms       3 ms  192.168.1.1
 2      47 ms      59 ms      30 ms  172.24.236.132
 3       *       *       *  Превышен интервал ожидания для запроса.
 4       *       *       *  Превышен интервал ожидания для запроса.
 5      54 ms      47 ms      43 ms  195.50.15.56
 6      38 ms      49 ms      29 ms  185.11.76.56
 7      38 ms      39 ms      49 ms  185.11.76.26
 8      57 ms      38 ms      39 ms  185.11.78.8
 9      56 ms      49 ms      59 ms  80.64.101.212
10       *       *       *  Превышен интервал ожидания для запроса.
11      61 ms      53 ms      52 ms  80.64.101.217
12      56 ms      49 ms      49 ms  185.65.148.253

Трассировка завершена.
```

Рисунок 10